

Universal Sovereign Identity

C. Kusuma

c.kusuma@proton.me

<https://sphinxfnd.org>

ABSTRACT

Privacy in a narrower scope is about proof of ownership of digital currency, or simply hiding transactions from public auditing — but privacy in the general sense is about something far greater and more fundamental: our digital life, our identity, our presence, and our ability to prove who we are within a universal proof layer for digital sovereignty.

At the center of this challenge is entropy — the source of unpredictability in cryptography, traditionally used to secure blockchain transactions. This work argues that entropy is not just a tool for securing currency, but a fundamental mechanism for ensuring verifiable proof of presence in a future shaped by these technologies.

Quantum computing (QC) and artificial intelligence (AI) are advancing simultaneously and at an accelerating pace; as these two forces collide, they form a technological tsunami that threatens both humanity itself and the cryptographic foundations underlying all of our digital systems and the trust they depend upon.

While QC challenges the mathematical basis of classical cryptography, AI introduces a parallel threat by enabling large-scale impersonation and accelerating attacks that weaken existing assumptions. As this new threat model moves beyond the limits of classical security frameworks, the need for a universal proof of identity becomes increasingly urgent.

Contents

1. Introduction	4
2. Digital Identity.....	4
3. Fingerprint.....	5
4. Data Integrity	6
5. Digital Signature	7
5.1 Hash Sensitivity	7
5.2 The .usimeta Container	8
6. Data Privacy and Sharing.....	8
7. Vault.....	10
8. Blockchain	12
8.1 To Embed	12
8.2 To Retrieve and Verify.....	12
9. Anchoring Phase.....	13
10. Computation Steps.....	13
10.1 Compute Raw-Data Hash	13
10.2 Assemble Metadata	13
10.3 Hash and Sign Metadata	13
10.4 Construct Transaction Payload	13
10.5 Broadcast Transaction.....	13
11. Retrieval and Verification Phase	14
11.1 Recalculate Hash of Retrieved Data	14
11.2 Verify Integrity	14
11.3 Verify Metadata Signature	14
12. Consensus	14
13. Consensus View	15
14. Intersection Guarantee	15
15. Fault Tolerance.....	15
16. Native Currency.....	16
17. Parameter Tables and Fee Formula	16
17.1 Denominations.....	16
17.2 Network Base-Rate Parameters.....	16
17.3 Utilization, Oracle, and Monetary Parameters	16
17.4 Conversions.....	17
18. Transaction Fee	17

19. Metadata Anchoring Fee.....	17
20. Smart-Contract Fee (Deploy / Execute)	17
21. Off-Chain IPFS Pinning (Oracle-Driven)	17
22. Total Quoted Costs.....	18
23. Dynamic Bounded Base-Rate Adjustment (Elasticity).....	18
24. Fee Distribution (Split)	18
25. Inflation Schedule	19
26. Governance — Starting Parameters.....	19
27. Conclusion.....	19
References	20

1. Introduction

Every invention came from a problem that we experienced. Satoshi Nakamoto invented Bitcoin when the world was reeling from a financial crisis — as a response to broken trust in centralized finance. Today, the problem is quite different. The challenge of our lifetime is no longer just who controls "money" — it is who controls "identity," and whether authenticity can survive at all.

Universal Sovereign Identity (USI) is based on a simple but powerful idea: owning a cryptographic key means owning your digital self. It treats identity not as something issued or controlled by institutions, but as something individuals hold and manage directly. In this sense, possession of a self-custody key becomes the foundation of personal digital sovereignty.

This idea goes further — the key is not only used to sign transactions as seen in traditional systems, but becomes a universal proof of authorship, intent, and existence across all digital interactions. Whether creating content, sending messages, or interacting with systems, the same cryptographic root can be used to verify that an action genuinely comes from a real, consistent identity.

This shifts the role of identity in the digital world. Instead of being tied primarily to economic participation, identity becomes a broader concept — one that defines presence, authenticity, and accountability. Individuals are no longer just participants in financial networks, but active agents who can prove who they are without relying on centralized platforms or intermediaries.

Redefining identity as cryptographic sovereignty, USI enables people to take control of their digital presence. It reduces dependence on trusted third parties and opens the door to systems where trust is derived from verifiable proofs rather than institutional authority. In a landscape increasingly shaped by AI-generated identities and post-quantum risks, this approach provides a path toward maintaining authenticity and human agency in the digital domain.

2. Digital Identity

Digital identity has traditionally been something issued to us — a passport number, a username, a government ID, or an account on a centralized platform. These systems serve important purposes, and they work well for what they were designed to do: confirming that a person matches a record held by an authority. But they were not designed to protect a person's digital life, nor to link identity to the digital content a person creates, owns, or shares. In each case, the identity belongs to the system that granted it, and the individual is merely a holder. It exists within that system's boundaries, and it carries no verifiable connection to anything the individual does or produces outside of it.

This system proposes a different foundation. Rather than relying on external authorities to issue and validate identity, it allows individuals and organizations to use their own native entropy — the

same randomness that secures their cryptographic keys — as the root of a verifiable identity. In other words, the very source of cryptographic strength that protects one's currency or assets can also serve as proof of who they are. Identity is no longer assigned; it is derived from something the user already owns and controls.

Cryptographically binding identity to user-generated randomness, this approach removes the need for centralized intermediaries in the verification process. There is no registry to breach, no single point of failure, and no dependency on any one organization's continued operation. Verification becomes a matter of mathematical proof rather than institutional trust. At the same time, privacy is preserved — the user reveals only what is necessary to prove a claim, without exposing the underlying entropy or unnecessary personal data.

This model does more than strengthen digital trust; it fundamentally changes the threat landscape. Fraud becomes significantly harder when identities cannot be fabricated or stolen from a central database. Data breaches lose much of their value when there is no honeypot of personal information to harvest. Identity exploitation — from impersonation to synthetic identity fraud — is mitigated at the root, because authenticity is tied to something that cannot be copied or forged. In a digital landscape where security threats evolve constantly, this forms a resilient shield for both personal and financial data.

Crucially, this approach also makes identity *linkable to content*. A traditional identity proves who you are to a system, but it cannot prove that a specific document, message, or file genuinely came from you. Entropy-based identity can. The same root that establishes who someone is can also sign and verify what they create, establishing a continuous, verifiable thread between a person and their digital output. This is the feature that existing identity systems lack — not because they are flawed, but because they were built for a different purpose.

Ultimately, digital identity becomes a self-sovereign construct: verifiable by anyone, controlled by no one but its owner, privacy-preserving by design, and resistant to manipulation. It is not merely a better way to log in or prove who you are — it is a redefinition of what identity means in a world where trust can no longer be assumed, only proven.

3. Fingerprint

The fingerprint acts like Alice's digital identity. It can be used to receive, transfer, and verify raw data from anyone who knows Alice's fingerprint.

- Unique hash of the public key (64 hex characters, spaced in groups of four).
- Identifies a public key without sharing the full key.
- Shared safely with others (e.g., Bob) for verification, signing trust, or vault recipients.

- Prevents impersonation — proves ownership via keypair match.

When Alice shares her fingerprint, others can use it to confirm that any signature, vault, or encrypted message claiming to originate from her indeed matches her keypair. This eliminates the need to reveal her entire public key or depend on centralized identity-verification systems.

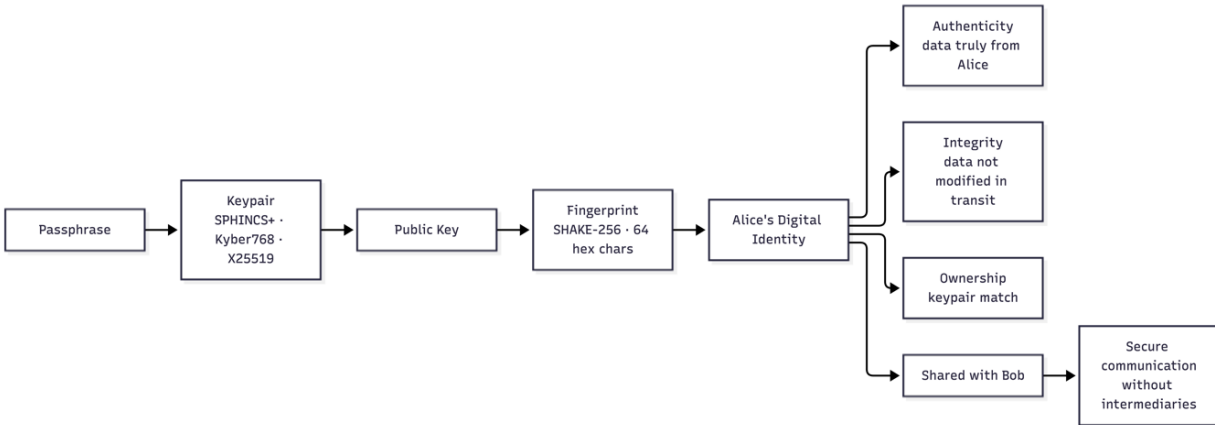


Figure 1. Fingerprint derivation from a public key.

In distributed or peer-to-peer environments, the fingerprint becomes a root of trust, allowing participants to establish secure communication and data exchange without intermediaries. It provides both authenticity (verifying that data truly came from Alice) and integrity (ensuring that the data was not modified in transit).

4. Data Integrity

Data integrity is the foundation upon which any trustworthy digital system is built. Without a reliable way to confirm that information originates from a legitimate source and remains unaltered, every transaction, record, and decision built on top of that data becomes suspect. In an era where AI can generate convincing false content and malicious actors can silently modify records, the question of *where data comes from* and *whether it can be trusted* has become urgent.

To address this, we propose binding a digital signature directly to the owner of a fingerprint. Rather than treating identity and data as separate concerns, this approach unifies them: the same unique, verifiable marker that proves who someone is also proves that the data they produce genuinely came from them. The fingerprint — already a deeply personal and difficult-to-replicate identifier — becomes the anchor for cryptographic proof, ensuring that every piece of data carries an unbreakable link to its origin.

This linkage ensures that every transaction or record carries verifiable proof of authorship and authenticity. When data is signed in this way, anyone can independently confirm its source without needing to trust a third party. If even a single byte is altered after signing, the signature fails, and the tampering is immediately exposed. This creates a system where trust is not assumed but

mathematically demonstrated — and where the integrity of the entire record is protected by the same mechanism that proves who created it.

Beyond individual transactions, this model establishes a broader principle: data should be self-verifying. Each record becomes its own proof of authenticity, carrying its origin and integrity within itself. In doing so, it removes the need for centralized verification authorities, reduces the attack surface for fraud and manipulation, and lays the groundwork for systems where authenticity is a built-in property rather than an afterthought.

5. Digital Signature

Each A digital signature is more than a technical mechanism — it is a declaration of origin. It answers a fundamental question: *who authorized this, and can we prove it?* In traditional systems, that proof depends on certificates, authorities, and infrastructure that can be compromised, revoked, or forged. In this system, each signature becomes an immutable proof of authorized origin within a decentralized network of verifiable data — self-contained, independently verifiable, and impossible to repudiate.

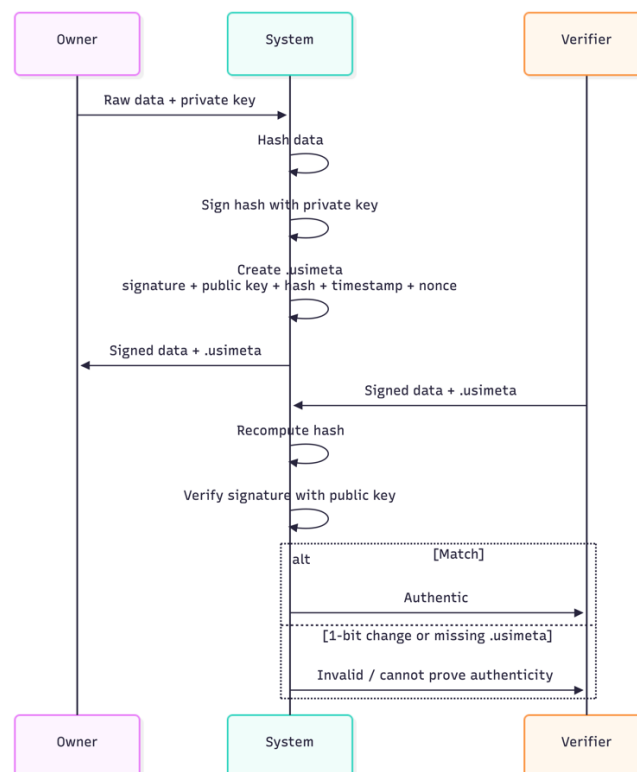


Figure 2. Signature embedding and verification flow.

5.1 Hash Sensitivity

Even a one-bit change in the input produces a completely different, unpredictable hash value. For example, changing “Hello World” to “Hello World!” changes the hash entirely:

- Original: a1d0c6e8...
- Modified: f1d2d2f9... (completely different)

5.2 The .usimeta Container

The .usimeta extension contains the cryptographic verification data. Without it, the system cannot prove the raw data’s authenticity.

```
{
  "Signature":      "hex_encoded_signature",    // Cryptographic signature
  "PublicKey":      "hex_encoded_public_key",    // Needed for verification
  "FileHash":       "original_file_hash",        // Reference for comparison
  "FinalDocumentHash": "final_hash",            // For PDFs with a signature page
  "Timestamp":      1234567890,                 // When signed
  "Nonce":          "random_value"              // Security nonce
}
```

6. Data Privacy and Sharing

The USI GUI separates local protection from recipient sharing. On the Message screen, Alice selects a folder, confirms her local session passphrase, and may enter one or more comma-separated recipient fingerprints. A blank recipient field creates a self-only personal vault; entering recipients starts the verified sharing flow.

- Before encryption, each recipient fingerprint is normalized, duplicates are removed, and every fingerprint is resolved through the local key directory to a registered HybridPublicKey bundle. The bundle binds the recipient identity fingerprint to its X25519 and Kyber768 public keys. If a fingerprint is invalid or cannot be resolved, encryption stops and no shared vault is created.
- For a shared vault, the backend creates a random 32-byte session key, archives the folder, computes a SHAKE256 hash of the plaintext archive, and signs that hash with Alice’s local SPHINCS+ signing key. The session key is then wrapped separately for each resolved recipient using an ephemeral X25519 exchange and Kyber768 encapsulation (when the Kyber public key is available); the combined secret protects the per-recipient AES-256-GCM key wrap.
- The folder payload is encrypted with the random session key using AES-256-GCM. The V3 manifest records the sender’s public key and signature, file metadata, encrypted-payload checksum, and one recipient entry per authorized fingerprint. Each recipient entry contains the X25519 ciphertext and, when used, the Kyber768 ciphertext. The resulting folder.vault file is the encrypted payload plus its manifest, integrity data, and recipient-specific key-wrapping material.
- An optional secure message is written temporarily as .encrypted_message.txt inside the selected folder so it is included in the encrypted archive; the temporary plaintext file is removed after the

encryption attempt. The original folder is preserved during encryption and removed only after the vault is successfully written.

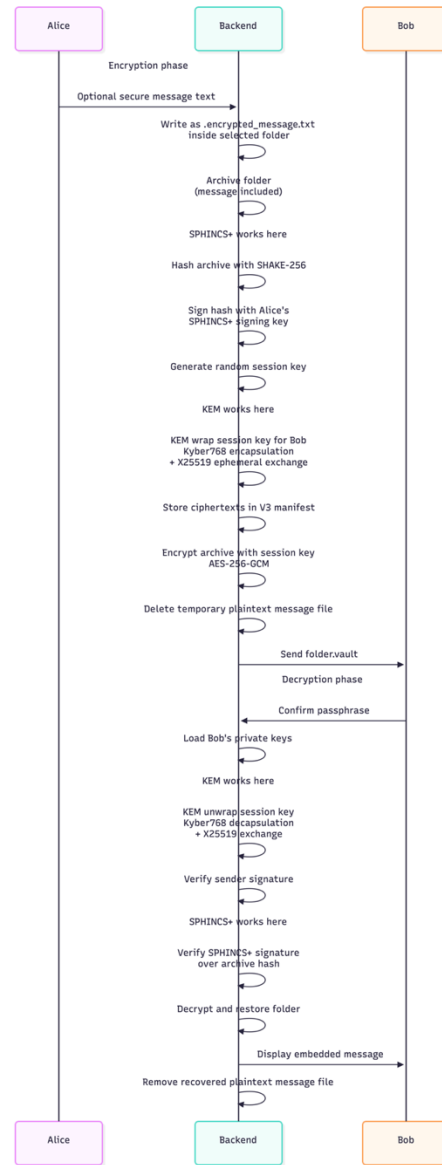


Figure 3. Local protection and recipient-sharing flow.

A self-only vault follows the GUI's no-recipient path and uses the V2 compatibility format. The backend derives a master key from Alice's passphrase and a random salt, generates a random session key, wraps that session key for the owner using the owner fingerprint, and encrypts the payload with AES-256-GCM. This path does not use per-recipient Kyber wrapping — Kyber/X25519 recipient wrapping applies only to the V3 sharing path.

Peer delivery is optional. If Alice provides exactly one recipient fingerprint and a peer address, the GUI sends the completed .vault file through USI-MAIL. The mail service treats the vault as opaque:

it does not encrypt, decrypt, or inspect the contents. It validates the envelope, recipient, .vault filename, size, and SHA-256 delivery hash, then stores the file in the recipient’s mail directory.

When Bob opens the Inbox screen, the GUI reads the vault manifest and derives Bob’s local fingerprint from the key protected by his passphrase. It displays whether the vault is personal, whether Bob is an authorized recipient, the sender fingerprint, and the recipient count — an authorization preview, since the encrypted payload has not yet been opened.

To unlock the vault, Bob confirms his passphrase. The backend loads the local signing/KEM key material, requires the local fingerprint to match a recipient entry (unless the vault is personal), unwraps the session key using the matching V3 private keys or the V2 compatibility mechanism, verifies authenticated encryption and vault integrity, and restores the original folder.

- If the folder contains an optional embedded message, the Inbox displays it after successful decryption and removes the recovered plaintext file. An incorrect passphrase, a fingerprint mismatch, a missing recipient key, a modified manifest, a failed MAC/checksum, or corrupted ciphertext causes decryption to fail rather than granting access.
- The security boundary is explicit: confidentiality comes from the vault’s authenticated encryption and recipient-specific key wrapping; authenticity comes from the sender’s signature over the plaintext archive hash; peer delivery provides transport and file-integrity checks only. The key directory and peer service are optional operational components, not trusted decryption authorities — the cryptographic access decision is made locally by the vault backend.

7. Vault

In traditional blockchain systems, what we think of as a “vault” address is usually very similar to a “wallet” address: a destination for sending and receiving cryptocurrency, with most interaction revolving around financial transactions. In this model, users rely on entropy-derived keys to unlock or reconstruct their private keys, which are then used to sign transaction data — in practice, the system is mostly focused on proving ownership of funds and authorizing transfers from one address to another.

This approach is limited in scope: the role of the private key is tightly bound to currency, and its use is largely restricted to signing transactions that move value across the network. While secure, it doesn’t fully take advantage of what cryptographic keys — and the entropy behind them — are actually capable of.

The idea of a vault expands this concept into something far more flexible. Instead of being limited to financial transactions, a vault allows users to decrypt their keys and use them to sign any type of data — messages, identities, credentials, proofs, or any digital interaction requiring verification.

In other words, the vault is not just a tool for moving money; it becomes a tool for expressing and proving ownership, intent, and authenticity in a much broader sense.

This shift also changes how we think about trust in digital environments. Instead of relying on centralized systems to validate identity or authorize actions, the vault enables users to generate their own cryptographic proofs directly. Every interaction — accessing a service, verifying a claim, establishing identity — can be backed by signatures derived from the same underlying entropy. This creates a model where trust is not granted by intermediaries but constructed through verifiable evidence, making cryptographic proof a native part of everyday digital interaction rather than something limited to financial use cases.

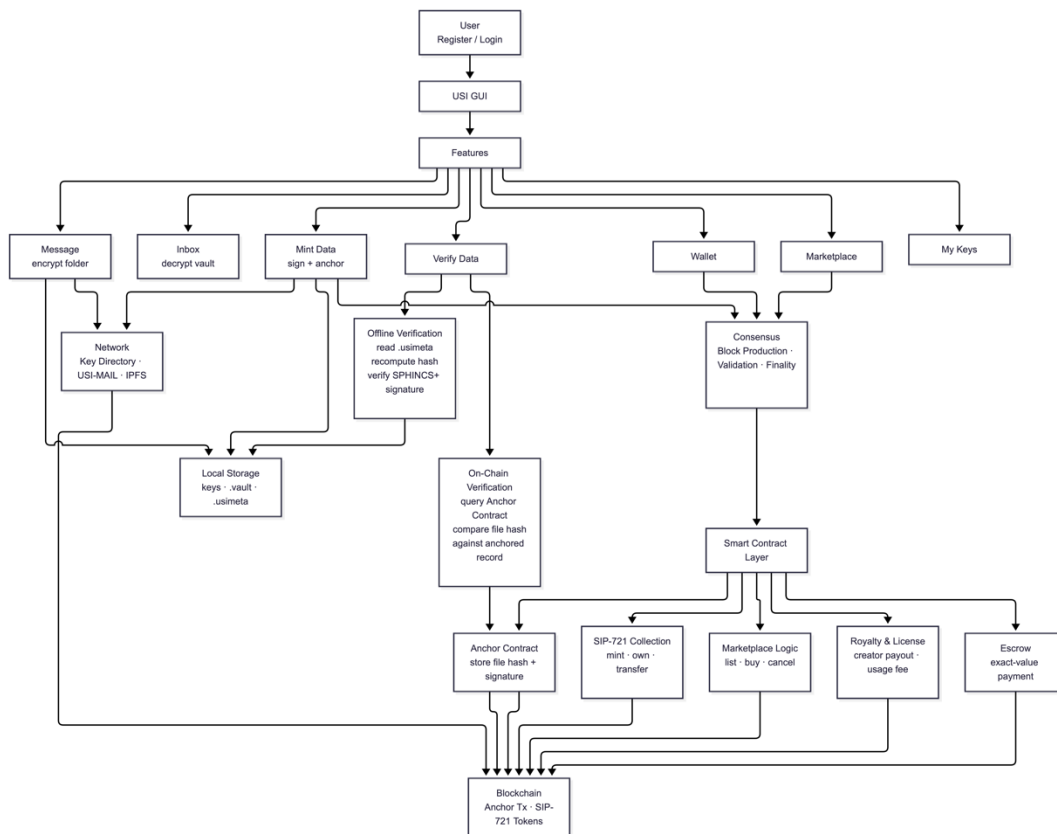


Figure 4. USI components mapped to familiar wallet primitives.

In this model, the components of a USI can be understood in a way that feels familiar to users of traditional blockchain systems. The USI passphrase functions much like the recovery phrase or seed phrase in a typical wallet — the core secret derived from entropy that allows a user to reconstruct and access their underlying keys. As long as the passphrase is preserved, the identity can always be recovered, regardless of device or environment.

At the same time, the USI fingerprint plays a role similar to a wallet address: a public-facing identifier others can use to reference, verify, or interact with the user. However, unlike a standard wallet address used primarily for receiving funds, the fingerprint represents a broader identity

endpoint — used to verify signatures, associate proofs, and anchor a wide range of interactions beyond financial transactions.

By doing this, the vault shifts from a simple transaction mechanism into a general-purpose cryptographic layer. The same entropy that once only secured financial value can now secure identity, actions, and presence in digital systems — opening the door to a model of digital ownership where users are defined not only by their balance, but by their ability to generate verifiable proofs across different contexts.

8. Blockchain

The blockchain is not designed to store large amounts of raw data directly. Instead, it functions best as a trust anchor — a permanent, verifiable ledger for cryptographic metadata rather than entire raw data.

In this model, only essential data such as hashes, content identifiers (CIDs), and digital signatures are written on-chain. The actual raw data or datasets remain stored externally in decentralized storage systems such as IPFS (InterPlanetary File System).

For example, a user can store a raw data (such as a PDF) in IPFS, which returns a unique CID derived from the raw data's content. This CID serves as a tamper-evident fingerprint of the raw data. The CID, along with optional metadata such as timestamps, version numbers, or signers' fingerprints, can then be hashed together and embedded into a blockchain transaction — minimal on-chain data linking the blockchain record to off-chain content without revealing or storing the raw data itself.

8.1 To Embed

- Compute the hash of the raw data or CID (e.g., SHA-256).
- Include the hash within the blockchain transaction's metadata or data field.
- Broadcast and confirm the transaction on-chain.

8.2 To Retrieve and Verify

- Obtain the CID from the blockchain record.
- Fetch the corresponding raw data from IPFS or other decentralized storage.
- Recalculate the hash of the retrieved raw data.
- Compare it against the hash stored on-chain to verify authenticity and integrity.

This design allows the blockchain to serve as an immutable proof layer for any off-chain content — ensuring that even though the data itself is stored elsewhere, its authenticity and origin remain verifiable on-chain.

9. Anchoring Phase

Let:

Symbol	Definition
D	raw data (e.g., PDF raw data)
$H(\cdot)$	cryptographic hash function (e.g., SHA-256)
$CID = H(D)$	IPFS content identifier derived from D
$M = \{CID, t, v, FP\}$	metadata set, where t = timestamp, v = version number, FP = signer's fingerprint (optional identity binding)
$H_M = H(M)$	hash of metadata
$\Sigma_{sk}(\cdot)$	digital signature function using private key $priv$
$Sig = \Sigma_{sk}(H_M)$	signature of metadata
Tx	blockchain transaction data field

10. Computation Steps

10.1 Compute Raw-Data Hash

$$CID = H(D)$$

10.2 Assemble Metadata

$$M = \{CID, t, v, FP\}$$

10.3 Hash and Sign Metadata

$$H_M = H(M), \quad Sig = \Sigma_{sk}(H_M)$$

10.4 Construct Transaction Payload

$$Tx_{data} = \{CID, H_M, Sig\}$$

10.5 Broadcast Transaction

Submit TX_{data} to the blockchain ledger. Once confirmed, it is permanently recorded at block height B_h .

11. Retrieval and Verification Phase

Given the blockchain record TX_{data} and the external raw data D' retrieved from IPFS via its CID:

11.1 Recalculate Hash of Retrieved Data

$$CID' = H(D')$$

11.2 Verify Integrity

$$CID' \cong CID$$

If equal, the retrieved data's content matches the on-chain reference.

11.3 Verify Metadata Signature

Using the public key pub :

$$Verify_{pub}(Sig) \stackrel{?}{=} H_M$$

If true, authenticity and signer identity are confirmed.

12. Consensus

In a decentralized or open network where n is unknown and can change arbitrarily, we cannot rely on fixed node counts to guarantee intersections as in classical BFT. Instead, protocols use relative or probabilistic measures.

If there are too few honest nodes, dishonest participants can outvote them. To keep truth intact, the number of honest participants must always outweigh the dishonest ones in any vote or decision round. If up to f nodes might be broken or dishonest, the system requires a quorum fraction Q , typically $\geq 2/3$ of all participants, to guarantee agreement:

$$Q = 2f + 1 \geq 2/3 \text{ (for large } n\text{)}$$

So the “ $2/3$ quorum” is exactly the supermajority of honest nodes needed to guarantee intersection and safety. Because then, even if up to f nodes lie and another f are delayed or confused, the remaining quorum of $Q \cdot n$ honest nodes can see each other's messages and agree on the same block — malicious nodes cannot form a false majority, so consensus is maintained.

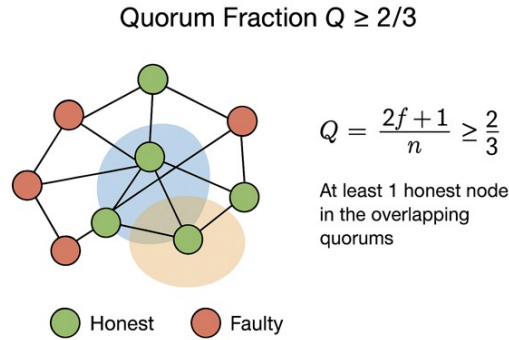


Figure 5. Quorum intersection under Byzantine fault assumptions.

Bitcoin solved this probabilistically with the longest-chain rule, also known as Nakamoto consensus. Let C be any candidate chain and $W(C)$ be its total accumulated proof-of-work. Honest nodes then adopt:

$$C^* = \arg \max_C W(C)$$

Security holds as long as honest miners control more than half of the global hash power, because an attacker with less than 50% cannot, in expectation, outpace the cumulative work of the honest majority.

13. Consensus View

Let n = total nodes and f = number of Byzantine (faulty or malicious) nodes.

To reach agreement, a quorum of $Q \cdot n$ nodes must approve a block or message, where $Q \geq 2/3$.

14. Intersection Guarantee

Any two quorums of size $Q \cdot n$ overlap in at least one honest node:

$$|A \cap B| \geq 2Q \cdot n - n = (2Q - 1) \cdot n > f$$

This ensures that at least one honest node sees both sets of votes and prevents conflicting blocks, even in dynamic networks.

15. Fault Tolerance

- The system remains correct as long as the fraction of Byzantine nodes does not exceed $1 - Q$.
- A quorum fraction $Q \geq 2/3$ guarantees safety, correctness, and finality, even with dynamic membership.

16. Native Currency

\$SPX is the native cryptocurrency of the ecosystem. Its model is designed to ensure long-term sustainability and to reward ecosystem participants.

- **Gas for the World Computer** — powers smart contracts and transactions.
- **Stake for the Truth Layer** — secures the network and pays for identity-related operations (anchoring, signing, vault creation), making it a “stake-for-trust” coin.

17. Parameter Tables and Fee Formula

17.1 Denominations

Unit	Definition
nSPX	Smallest unit (1 nSPX).
SPX	$1 \text{ SPX} = 10^{18} \text{ nSPX}$.
MaxSupply	$5 \times 10^9 \text{ SPX}$.

17.2 Network Base-Rate Parameters

All values are expressed in nSPX unless otherwise noted.

Symbol	Meaning
B_bw	Base bandwidth cost (nSPX / byte).
B_st	Base storage cost (nSPX / byte).
B_cmp	Base compute cost (nSPX / compute-op).
R	Replication factor (copies stored by validators).
α (alpha)	Hash / verification complexity factor (nSPX per hash).
β (beta)	Fixed verification overhead (nSPX).
K_tx	Fixed transaction overhead (nSPX).

17.3 Utilization, Oracle, and Monetary Parameters

Symbol	Meaning
Util	Current block utilization (0–1).
Target	Target utilization (e.g., 0.70).
PinRate	Oracle pin rate (SPX / GB / month).

Symbol	Meaning
d	Pin duration, in months.
Infl ₀	Initial inflation (annual fraction).
γ (gamma)	Inflation decay factor per year (e.g., 0.85).
BlockTime	Seconds per block.

17.4 Conversions

$$GB = \text{bytes} / 1024^3$$

$$SPX = nSPX / 10^{18}$$

18. Transaction Fee

Let S_{tx} = transaction size (bytes) and Ops_{tx} = compute operations required:

$$TxFee_{nSPX} = B_{bw} \cdot (S_{tx} \cdot R) + B_{cmp} \cdot Ops_{tx} + K_{tx}$$

Displayed as SPX via the conversion in §17.4.

19. Metadata Anchoring Fee

Let M = metadata size (bytes) and H = number of hashes / Merkle leaves:

$$SigFee_{nSPX} = B_{st} \cdot (M \cdot R) + \alpha \cdot H + \beta$$

Displayed as SPX:

$$TxFee_{spx} = TxFee_{nSPX} / 10^{18}$$

20. Smart-Contract Fee (Deploy / Execute)

Let C = estimated operation count and $S_{contract}$ = permanent contract storage (bytes):

$$ContractFee_{nSPX} = B_{cmp} \cdot C + B_{st} \cdot S_{contract}$$

21. Off-Chain IPFS Pinning (Oracle-Driven)

Let F = raw data size in bytes, d = months to pin, and $PinRate$ in SPX/GB/month:

$$IPFSFee_{SPX} = PinRate \cdot d \cdot (F / 1024^3)$$

If nSPX is required:

$$IPFSFee_{nSPX} = IPFSFee_{SPX} \times 10^{18}$$

22. Total Quoted Costs

Shown before sign metadata, for a user action that requires an on-chain component and an IPFS pin:

$$TotalFee_{nSPX} = OnChainFee_{nSPX} + IPFSFee_{nSPX}$$

Where OnChainFee is one of {TxFee, SigFee, ContractFee}.

23. Dynamic Bounded Base-Rate Adjustment (Elasticity)

Compute the multiplicative adjustment from utilization:

$$factor = 1 + (Util - Target) / 10$$

Clamp to $\pm 20\%$:

$$factor' = \max(0.8, \min(1.2, factor))$$

Apply to any base B:

$$B_{new} = B_{current} \cdot factor'$$

24. Fee Distribution (Split)

For any collected fee F_total:

Recipient	Share
Validators	60% of F_total
Stakers	25% of F_total
Treasury	10% of F_total
Burned	5% of F_total

25. Inflation Schedule

Initial annual inflation Infl_0 in year y (1-indexed):

$$\text{Inflation}(y) = \text{Infl}_0 \cdot \gamma^{(y-1)}$$

Approximate blocks per year:

$$\text{BlocksPerYear} = (365 \cdot 24 \cdot 3600) / \text{BlockTime} \approx 31,536,000 / 12 \approx 2,628,000$$

26. Governance — Starting Parameters

Parameter	Starting value
B_bw	20,000 nSPX / byte
B_st	200,000 nSPX / byte
B_cmp	5,000,000 nSPX / op
R	3
α (alpha)	10,000 nSPX / hash
β (beta)	200,000 nSPX
K_tx	1,000 nSPX
PinRate	0.01 SPX / GB / month
Infl_0, γ	5% annual, 0.8 decay
Target, BlockTime	0.70, ≈ 12 s

27. Conclusion

We proposed that USI's entropy — the cryptographic keys used to prove ownership of digital currency — should be extended far beyond its economic purpose. In this expanded model, entropy becomes not just a cryptographic primitive, but a personal source of truth: a self-derived identity that anchors both ownership and privacy.

This shift matters because it redefines what a key *is*. Traditionally, a private key is a tool for signing transactions — a functional instrument with a narrow scope. But once we recognize that the same entropy can prove authorship, establish presence, and verify authenticity across every digital interaction, the key transforms into something far more fundamental. It becomes the root of the

self in the digital world — a single, sovereign source from which identity, ownership, and trust all flow.

Every individual's entropy can generate a unique root of identity, one that no centralized or third party can replicate, predict, or forge. Because it is derived from the user's own randomness rather than issued by an authority, it cannot be revoked, seized, or duplicated. It exists independently of any institution, platform, or government. In this sense, entropy forms the core of digital dignity — protecting not only financial assets but also personal data, documents, and communications. It ensures that a person's digital existence remains theirs alone, not something lent to them by a system that can take it away.

Blockchain technology, under this model, becomes more than an economic ledger; it becomes an infrastructure of personal sovereignty, where privacy is mathematically enforced rather than merely promised. Trust is no longer a matter of policy or good faith — it is a property of the system itself. Verification replaces permission. Proof replaces authority. And the individual, armed with nothing more than their own entropy, stands as the ultimate guarantor of their own truth.

In a world increasingly shaped by AI-generated forgeries and quantum threats to cryptography, this is not merely a technical improvement — it is a necessary evolution. USI offers a path toward a digital future where authenticity is provable, identity is sovereign, and human agency is preserved not by the goodwill of institutions, but by the unbreakable logic of mathematics.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System." <https://bitcoin.org/bitcoin.pdf>
2. Ethereum Foundation, "Ethereum Whitepaper." <https://ethereum.org/whitepaper/>
3. L. Lamport, R. Shostak, M. Pease, "The Byzantine Generals Problem." <https://lamport.azurewebsites.net/pubs/byz.pdf>
4. Wikipedia, "Digital Identity." https://en.wikipedia.org/wiki/Digital_identity
5. Wikipedia, "Public Key Fingerprint." https://en.wikipedia.org/wiki/Public_key_fingerprint
6. Wikipedia, "Hash Chain." https://en.wikipedia.org/wiki/Hash_chain
7. Wikipedia, "Byzantine Fault." https://en.wikipedia.org/wiki/Byzantine_fault